

LicenseToken-2026: On-Chain Software Licensing as a Native Asset

Quantum Discoveries
quantumdiscoveries@proton.me

Abstract

A purely on-chain form of software licensing would let the right to execute a program be sold directly from its creator to its user, with no server, no company, and no trusted enforcer in between. Digital signatures provide part of the answer, but most of the benefit is lost if a trusted third party is still needed to check whether someone is allowed to run the software. We propose a way to enforce software licensing using the Solana runtime itself. A license is recorded as an account at a deterministic address. A program consults that account before it runs its privileged logic, and the chain, which already decides what is true, decides who is permitted. Payment, expiry, renewal, gift, sale, and revocation are ordinary program instructions that settle without an intermediary. The protocol charges no fee: every payment, and every secondary-sale royalty, goes in full to the creator, or to a permanent founder wallet the creator themselves designates, and there is no protocol treasury and no privileged account. When a creator gives up the authority to upgrade their program, the licensing rules become permanent, so the user is protected from the creator as much as the creator is protected from the user. The result is a fourth native asset type. Not spendable value, not fungible ownership, not unique ownership, but the right to execute.

1. Introduction

Software on the internet relies almost entirely on servers and companies to enforce the right to use it. The model works well enough most of the time, but it carries the usual weaknesses of trusting a third party. Enforcement lasts only as long as the server that performs it and the company that runs it. Terms can change after purchase. Access can be withdrawn for reasons that have nothing to do with the agreement. A user has no real assurance that the software they licensed today will behave the same tomorrow, because the party enforcing the license is also the party able to change it.

On a blockchain, that trusted enforcer was supposed to disappear. So far it has not. Where on-chain software is monetized at all, it is usually monetized by launching a speculative token or by charging a flat fee per call. Neither is licensing. Neither expresses a subscription, a tier, a trial, a renewal, a resale, or a royalty, which is the ordinary grammar of the software business. Working code, arguably the most valuable thing built on-chain, has had no native way to be sold as what it is.

What seems to be missing is a licensing system based on cryptographic enforcement rather than trust, so that two parties can transact the right to run a program without relying on a third. We propose such a system. The right to execute becomes an asset, the program becomes its own enforcer, and once the creator chooses to give up that ability, no one can change the terms after the fact.

2. The License

A license is an account that records one wallet's right to run one program until one moment in time. It holds the holder, the program it applies to, the tier purchased, the time of issue, and the time of expiry. An expiry of zero means a perpetual license that never lapses.

The account lives at a Program Derived Address, a deterministic location computed from the configuration it belongs to and a sequential issue number. Because the address is derived rather than chosen, a license cannot be forged. Only the program, running its own logic, can create one, and only after payment has settled.

The license is the asset. It is the thing that is bought, held, renewed, transferred, revoked, and checked. Everything else in the protocol exists to issue it, price it, or honor it.

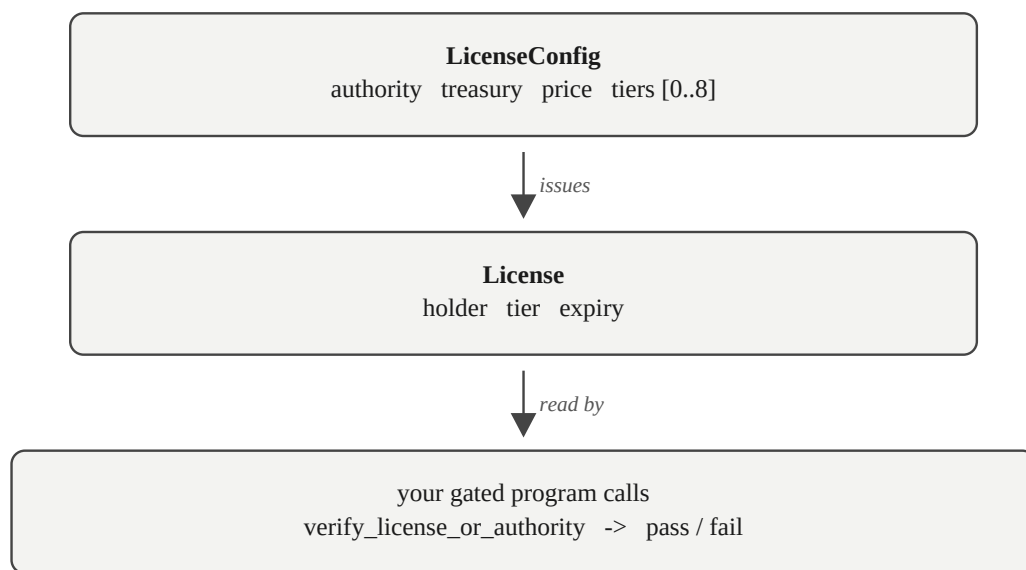


Figure 1. Three accounts. The config prices it, the license proves it, and the gate checks it inside your program.

3. The Configuration

Each licensed program has one configuration, which is the creator's rulebook. It records a flat price and period, whether licenses may be transferred, the royalty owed on a secondary sale, an optional cap on total supply, an optional grace period, and where payment should go. It can also define up to eight tiers. Each tier is a small rulebook of its own, with its own price, its own period (its own expiry), its own supply cap, and its own state, which is active, retired, or paused. A retired tier sells no new licenses but still lets existing holders renew. A paused tier freezes both. Tiers can be added and amended over the life of a product without recreating the configuration. Amending a tier does not change the tier or expiry of any license already issued into it, but renewal is always charged at the tier's current price, so a holder who renews after a price increase pays the new price. This is a deliberate tradeoff: creators keep the ability to adjust pricing over time, and buyers know that renewal price is subject to the current

configuration.

Only the program's upgrade authority can create a configuration. The protocol checks this directly. It reads the program's account under the BPF Loader, finds the program data record, and confirms that the signer is the current upgrade authority and that the authority has not been burned. Someone who does not control a program cannot license it. This one check is the source of every license's legitimacy.

The configuration's address is derived from the program alone. There is exactly one configuration per program, and anyone can compute its location from the program's identity, without a search and without knowing who created it. The owner is recorded inside the configuration rather than encoded in its address, so control can pass to a new owner without the configuration ever moving.

It helps to name the roles, because over time they can come apart. The creator is whoever builds and registers the program. Control lives in the configuration's authority, which the creator holds at first but can transfer or give up. Revenue flows to the treasury, which the authority can reappoint. And a creator may set a founder, a permanent payout wallet that no later owner can change. At the start a single person is all of these at once. After a sale they can be different parties, and the rest of this paper is careful to say which one it means.

4. Enforcement

A license does nothing until a program agrees to honor it. The protocol gives the program one operation for that:

```
verify_license_or_authority(license, config, config_key, caller, program_id)
```

It succeeds if the caller holds a valid license, meaning the holder matches the caller, the program matches, the license is active, and it has not expired beyond its grace period, or if the caller is the configuration's current authority. Whoever holds that authority always passes, on the simple principle that if you control it, it is yours. At first that is the creator; after a hand-off it is the new owner, and the original creator no longer passes unless they also hold a license. A founder, which is only a payout wallet, gets no access from this check at all. A stricter operation, `verify_license`, drops the authority exception and admits no one without a valid license, while still honoring the grace period.

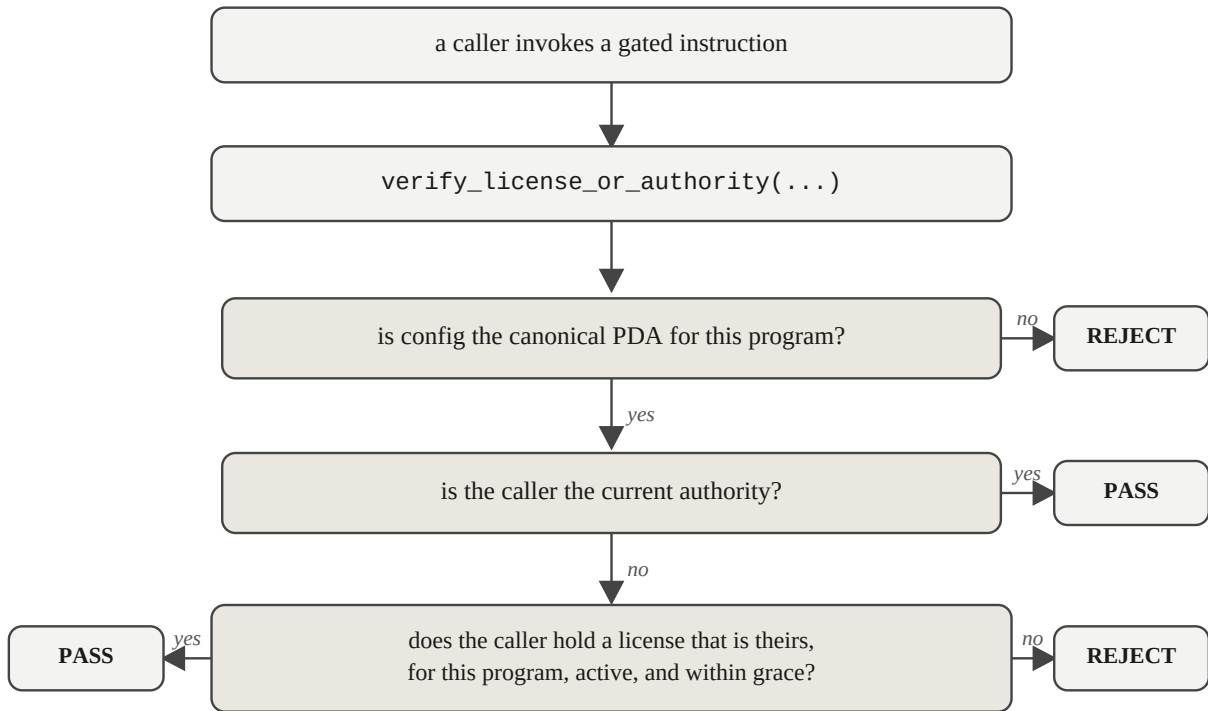


Figure 2. The gate decides who may execute. A failed check is not an error to handle later; the transaction simply does not run.

The operation defends itself. Before it trusts any field on the configuration it is given, it recomputes the canonical configuration address for the program and rejects any other account, so a caller cannot hand in a configuration of their own making to spoof authority or stretch a grace period. The integrator is protected even if they forget to constrain the account themselves.

The check runs inside the program's own instruction, before any privileged logic. Enforcement is not advisory and not external. The program performs it as a mandatory step in its own execution path, and the Solana runtime executes that path faithfully. Once the upgrade authority is burned and the program is frozen, that path can never be removed. Before the burn, the creator still controls the code and the guarantee is one of intent, not immutability. There is no server to bypass and no request to intercept, and a transaction that fails the check does not run. Every state-changing instruction also emits a structured event, so the full history of a license, minted, renewed, gifted, sold, revoked, can be reconstructed by any observer without a private index.

5. Settlement

Payment is an instruction. To subscribe is to call `mint_license`. The caller pays the configured price, either the flat price or the price of the chosen tier, and a license is issued to their wallet in the same transaction. The whole payment stays on the creator side and nothing is taken by the protocol. By default it all goes to the treasury; if a founder is set, it splits between the founder and the treasury, as described below. A payer can also mint to another wallet, so a license can be a gift, a trial, or a grant to a program.

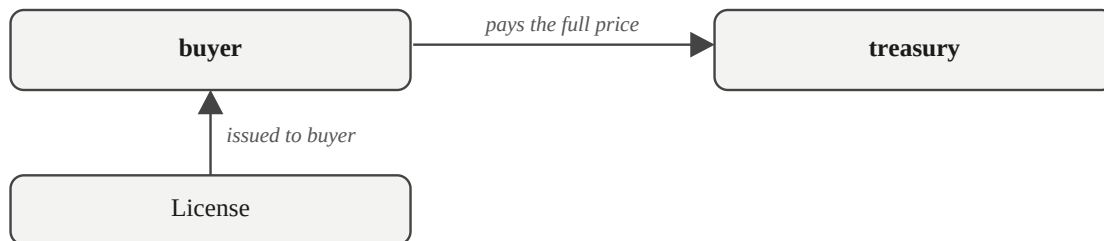


Figure 3. The base case, with no founder set: the whole price goes to the treasury, and the protocol takes nothing. With a founder, it splits as in Figure 4.

The protocol takes nothing. There is no protocol fee, no protocol treasury, and no privileged account to set up, so there is nothing to front-run, drain, or trust. A neutral primitive that profits from no one's usage is easier for a whole ecosystem to adopt. The intent is that it earns its place by being useful, and that anyone wanting to make money does so by building on top of the standard rather than taxing it from within. There are no emissions and no subsidy. The only value that moves is the price a creator sets, paid in full to the creator.

More than that, it puts developers in control of their own economics. They set the price and keep all of it, they choose the terms, and they need no platform's permission to participate. This is the part worth dwelling on, because it is what crypto was meant to be for in the first place: not a new middleman, but the removal of one. The standard is a public good that gives builders more options and more control over their own work, a way to monetize what they make on their own terms, rather than a gatekeeper that grants access and keeps a cut.

A creator can go one step further and make part of that revenue permanent. Using `set_founder` they may name a wallet, once, together with a share. From then on that wallet receives its share of every creator-side payment, on a subscription, on a renewal, and on the royalty of a sale, for as long as the program runs. The choice is optional, and the wallet is written only once: no later edit, no new owner after a sale, and no act of renouncement can change it. A licensor can hand the whole project to someone else, treasury and all, and still keep their share for life.

The split is exact. The founder takes their configured share of the amount and the treasury takes the rest, defined as the remainder so the two always sum to the payment, down to the last lamport. Set the share to its maximum and the founder keeps the entire stream. Set it lower and an operator who takes over the treasury keeps the difference. Either way it is fixed in the open, so a buyer of the config can read exactly what they are getting before they pay.

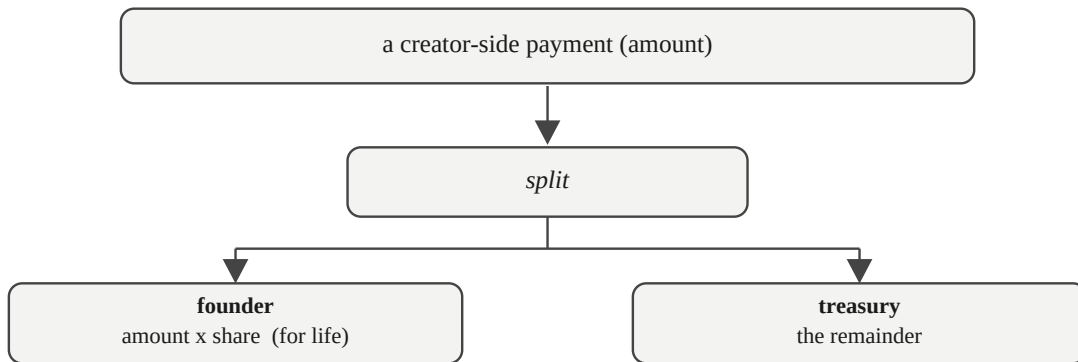


Figure 4. The founder's share is set once and outlives every later hand-off or renouncement of the config.

6. Immutability

The authority to upgrade a program is separate from the authority to manage its licensing. The first controls the code. The second controls the terms, meaning price, tiers, and revocation, and it can be transferred on its own.

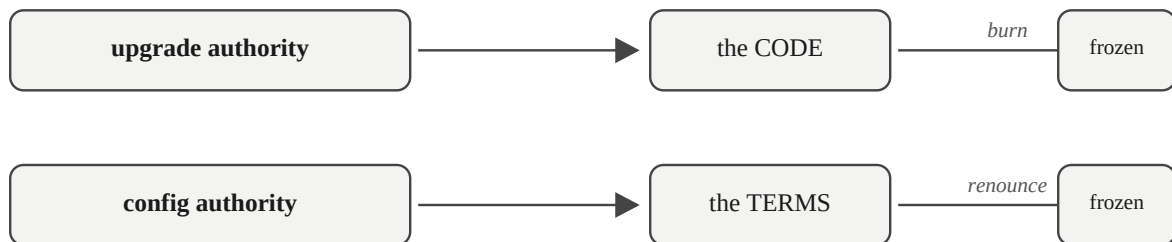


Figure 5. Two separate keys. Each can be given up on its own, and either choice is a public fact anyone can verify on-chain.

That separation matters. A creator can burn the upgrade authority, after which the code can never change. The license check is part of that code, so it can then never be removed, not by an attacker, not by a successor, not by a court order served on a company that no longer holds the keys, and not by the creator. Because licensing control is a separate authority, the creator can still set prices after the burn, while the rules of the gate move out of everyone's reach.

A creator can also give up the licensing authority itself, and can do it in degrees. They can give up only the power to revoke, which is a one-way change, after which no license can ever be cancelled, while pricing and tiers stay adjustable for future buyers. Or they can give it up entirely, by setting the authority to an address with no private key, after which the configuration is frozen: price, tiers, treasury, and revocation fixed for good. Each step is a plain, public fact on-chain, so a prospective buyer can check, before paying, exactly which powers the creator has surrendered, instead of relying on a promise.

It is worth being honest about the default. Before the upgrade authority is burned, the creator can still redeploy the program, which means the license gate exists as a matter of intent, not cryptographic necessity. After the burn, it is necessity. A creator who keeps the upgrade authority but not the licensing authority still controls the code; a buyer of the licensing authority should understand this window exists until a burn is verified on-chain. A creator who keeps the licensing authority can still change future terms, much as an off-chain vendor can. The difference is twofold. Every such change is a public record the buyer can see and weigh, not a silent edit on a private server. And the power to make changes can itself be given up for good, and then verified to be gone. What a buyer already holds is safer still, because an issued license's expiry and tier are fixed at the moment of mint and lie beyond the creator's reach.

This makes a kind of assurance that is hard to offer off-chain. A license bought against an immutable program will be honored as written for as long as the chain exists, because no one keeps the power to change it. The usual model protects the vendor from the user. This one also protects the user from the vendor. Less has to be trusted, because more can be verified.

7. Gift and Sale

A transferable license can change hands in two ways, and the protocol keeps them apart.

A gift (`gift_license`) is a plain reassignment. The holder hands the license to another wallet and nothing is paid. It is how you move a license between your own wallets, or give one away.

A sale (`sell_license`) is a priced, atomic transfer that needs no trust. The buyer and the seller co-sign a single transaction. The buyer pays the agreed price, the protocol takes the creator's royalty from that price and routes it to the creator side, the founder and the treasury, the remainder goes to the seller, and the license moves to the buyer, all at once. Neither side can take the other's value without giving up their own, because it is one transaction or none. The royalty is enforced by the instruction itself, not by a marketplace's goodwill, and all of it stays on the creator side.

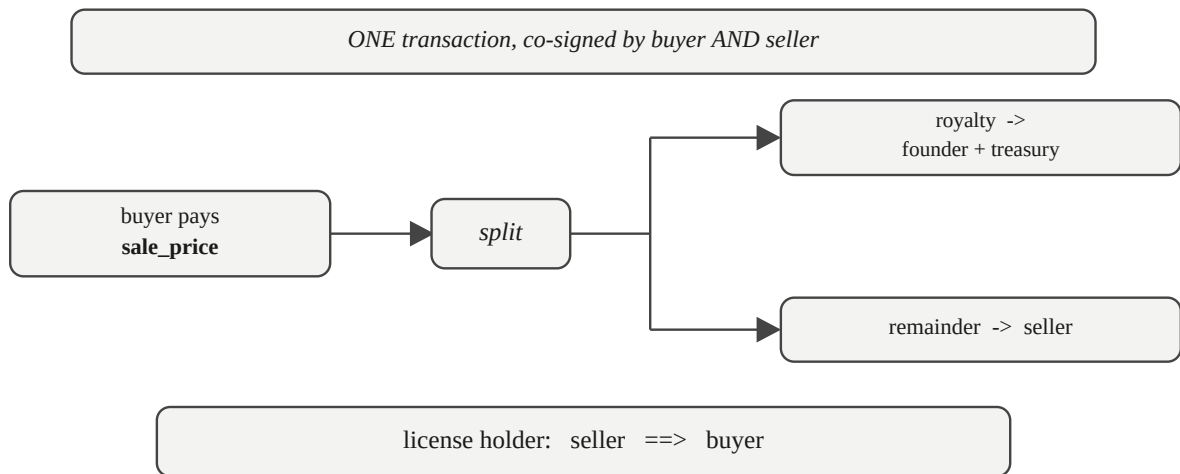


Figure 6. The payment and the change of holder happen together, or not at all. The royalty is creator-side and splits between founder and treasury; the seller's remainder is untouched.

So a license is a liquid asset. The unused remainder of a subscription can be sold, and the creator keeps earning on every resale. A configuration that is not transferable forbids both gift and sale.

8. Renewal, Expiry, and Reclamation

A subscription expires at its recorded time unless it is renewed. Renewal calls `renew_license`, which extends the expiry by the license's own tier period, at the current price, paid in full to the creator. Renewal belongs to the holder. Only they can extend their license, and a creator can never reach in to shorten an issued expiry, change a license's tier, or shrink its grace window, because all three are fixed at the moment of mint. A perpetual license, with a period of zero, needs no renewal and cannot expire. Renewal is allowed on an active or a retired tier, so sunsetting a plan never strands the holders left in it, while a paused tier suspends renewal until it resumes.

A holder who no longer wants a license can call `close_license`, which removes the account and returns the rent that paid for its storage. A creator can call `revoke_license`, which ends a holder's access the instant it is called. Supply can be capped, both globally and per tier, and the protocol refuses to set a cap below what has already been issued, so issuance can never be invalidated after the fact.

9. Composability

A license can be issued to a program as easily as to a person. The `mint_license_to` instruction separates the payer from the holder. One party pays, another holds, and the holder can be a Program Derived Address.

So one program can hold a license to another. Any payer can grant a license to a program-derived address, and that address becomes the holder. For ongoing access the simplest arrangement is a lifetime license, with a period of zero, which never expires and so never needs renewal. A program granted one

holds its access for good, with nothing further to do, and this is what we would recommend for most program-to-program use.

A time-limited license held by a program can still be renewed, but it asks for more care, and the honest details matter. Renewal is reserved to the holder, so the holding program must be written with an instruction that renews itself, signing for its own address with its seeds; no outsider can renew on the program's behalf. And because a program only acts when some transaction invokes it, the renewal happens when the program is run to do it, by a user, a keeper, or a scheduler, never on its own. Self-renewal is therefore the option for the cases that genuinely need access to lapse unless it is paid for; when in doubt, a lifetime license avoids the question entirely.

Either way, the right to execute, recorded as an account, can move between programs as readily as between people.

10. Scope

The protocol enforces the right to execute on-chain programs, where the runtime performs the check as a condition of running, and where the gate cannot be removed once it is frozen. Within that domain the enforcement is complete.

It does not reach software it does not run. A program that runs off-chain, on a private server, in a closed binary, or on a user's own machine, cannot be compelled by a chain that does not execute it. For that kind of software a license is a payment and a credential the operator can choose to honor, not a lock. This boundary is a property of the medium, not a shortcoming of the protocol, and it is worth stating plainly so that no one builds on a guarantee that was never offered. It is also no different from any token. SPL and Token-2022 cannot reach off-chain software either. What sets this apart is that, for on-chain programs, the license actually gates execution.

11. The Fourth Asset Type

A blockchain has so far produced three native asset types. The first is spendable value. The second is fungible ownership. The third is unique ownership. Each is a way to record a true statement about having, and each, once it became native, made an economy possible that could not exist before it.

None of them records a statement about using. Yet the largest businesses in software are built not on owning but on the recurring right to use, granted for a time and under terms. The right to execute is the fourth such statement, and until now it has had no native form.

The license described here is that form. It is not value, not a share, and not a unique object. It is permission, recorded as an asset, priced by its issuer, and enforced by the program it governs. It fills in the one that was missing.

12. Conclusion

We have described a way to do software licensing that does not depend on trust. We started with the asset, a license at a deterministic address, then defined the configuration that prices it, the operation that enforces it, and the settlement that pays for it, in full, to the creator. We showed that when a creator gives up the authority to change their program, and gives up the authority to change its terms, a license becomes permanent in a way a buyer can verify rather than trust, so the user is protected from the creator by the same mechanism that protects the creator from the user. We covered the renewal and revocation that govern a license's life, the lifecycle of tiers, the gift and the atomic sale that make a license a liquid asset with a creator royalty, and the composability that lets one program license another. The protocol holds no custody, controls no creator's keys, and charges no fee. It is meant to be a neutral standard rather than a tollbooth. The right to execute, long missing from the chain, becomes a native asset, and the economy of on-chain software can follow from it.

References

- [1] R. H. Coase, "The Nature of the Firm," *Economica*, vol. 4, no. 16, pp. 386-405, 1937. <https://onlinelibrary.wiley.com/doi/10.1111/j.1468-0335.1937.tb00002.x>
- [2] W. Diffie and M. E. Hellman, "New Directions in Cryptography," *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644-654, 1976. <https://ee.stanford.edu/~hellman/publications/24.pdf>
- [3] Free Software Foundation, "GNU General Public License, Version 1," 1989. <https://www.gnu.org/licenses/old-licenses/gpl-1.0.html>
- [4] N. Szabo, "Smart Contracts," 1994. <https://nakamotoinstitute.org/library/smart-contracts/>
- [5] N. Szabo, "Formalizing and Securing Relationships on Public Networks," *First Monday*, vol. 2, no. 9, 1997. <https://firstmonday.org/ojs/index.php/fm/article/view/548>
- [6] L. Lessig, "Code and Other Laws of Cyberspace," Basic Books, 1999. <https://lessig.org/product/code/>
- [7] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. <https://bitcoin.org/bitcoin.pdf>
- [8] F. Vogelsteller and V. Buterin, "EIP-20: Token Standard," *Ethereum Improvement Proposals*, 2015. <https://eips.ethereum.org/EIPS/eip-20>
- [9] A. Yakovenko, "Solana: A new architecture for a high performance blockchain," 2018. <https://solana.com/solana-whitepaper.pdf>
- [10] W. Entriiken, D. Shirley, J. Evans, N. Sachs, "EIP-721: Non-Fungible Token Standard," *Ethereum Improvement Proposals*, 2018. <https://eips.ethereum.org/EIPS/eip-721>
- [11] Solana Labs, "Token-2022 (Token Extensions) Program," Solana Program Library documentation. <https://spl.solana.com/token-2022>
- [12] Z. Burks, J. Morgan, B. Malone, J. Seibel, "EIP-2981: NFT Royalty Standard," *Ethereum Improvement Proposals*, 2020. <https://eips.ethereum.org/EIPS/eip-2981>